Project Documentation

Project Title: EventForce Management System

Team Members:

1. GIFTA K – Team Leader
   Reg. No: 920323243007
   NM ID: B6CABAA4ABB136E99B84AE2D1CA9FE04

2. DIVYA R – Team Member
   Reg. No: 920323243004
   NM ID: 0043B0DCC87171A802042B4CE077A14B

3. HEMALATHA V – Team Member
   Reg. No: 920323243009
   NM ID: 275F89AE67F5CC888FE4CBD45FAAFBF2

4. LIDHIYA SHYNI K – Team Member
   Reg. No:920323243014
   NM ID: 480EC4EDC7F2F81B66B448535F7A330A

1. PROJECT OVERVIEW

1.1 Executive Summary

The EventForce Management System is a Customer Relationship Management (CRM) solution built on the Salesforce Lightning platform within a Developer Edition Org. It is designed to streamline the complete lifecycle of event planning, from capturing a client enquiry and scheduling an event to coordinating vendors and venues, controlling budgets, and collecting post-event feedback.

Event planning organizations typically manage this information across spreadsheets, emails, and messaging applications. This scattered approach leads to duplicated records, missed deadlines, uncontrolled budgets, and poor visibility for management. EventForce replaces these fragmented tools with a single, secure, and automated platform in which every client, vendor, venue, event, and feedback record is stored, related, and reported on in one place.

The solution is delivered as a Lightning application named Event Planner. It contains dedicated tabs for Events, Clients, Vendors, Venues, Event Vendors, Feedbacks, Reports, and Dashboards. Business rules are enforced through validation rules, routine communication is handled by a record-triggered and scheduled Flow, high-value budgets are governed by an approval process, and complex backend logic is implemented with Apex triggers. Role-based security and live dashboards complete the solution.

1.2 Business Objectives

- **Central Data Management:** All client, vendor, venue, event, and feedback data is stored in a single system of record, eliminating duplicate and inconsistent information.
- **Automation:** Repetitive tasks such as event reminders, budget approvals, and status updates are performed automatically using Flow, Approval Processes, and Apex, reducing manual effort and human error.
- **Scalability:** The data model and security model are designed to support a growing number of events, users, and vendors without redesign, using declarative configuration wherever possible.
- **Operational Efficiency:** Validation rules, guided record pages, and automated notifications ensure that staff spend time on planning rather than on correcting data or chasing updates.
- **User Experience:** A clean Lightning interface, record-type specific page layouts, and visual dashboards give each user a focused and intuitive working environment.
- **Data Privacy and Governance:** Profiles, roles, permission sets, and sharing rules ensure that every user sees only the information appropriate to their responsibilities, particularly financial data.

1.3 Scope of the System

The scope of the project covers the following functional areas:

- Client management, including contact details and event history.
- Event scheduling with date, type, venue, budget, and status tracking.
- Vendor management and the assignment of vendors to specific events.

- Venue management and venue selection for events.
- Feedback collection and rating analysis after event completion.
- Automated reminders, budget approvals, and business rule enforcement.
- Role-based security, reporting, and executive dashboards.

1.4 Technology Stack

Component	Technology / Feature Used
Platform	Salesforce Lightning Experience (Developer Edition)
Data Layer	Custom Objects, Custom Fields, Master-Detail and Lookup Relationships
User Interface	Lightning App (Event Planner), Page Layouts, Record Types, Lightning Record Pages
Declarative Automation	Validation Rules, Flow Builder, Approval Processes, Email Alerts
Programmatic Logic	Apex Triggers, Apex Handler Classes, Apex Test Classes
Security	Profiles, Roles, Permission Sets, Organization-Wide Defaults, Sharing Rules
Analytics	Custom Report Types, Reports, Lightning Dashboards

2. PHASE-WISE IMPLEMENTATION DETAILS

2.1 Phase 1: Requirement Analysis and Planning

The first phase focused on understanding how an event planning business operates and translating those needs into Salesforce capabilities. Stakeholder needs were collected, grouped by business function, and each requirement was assessed to decide whether it could be met through standard features, declarative configuration, or custom code.

2.1.1 Key Business Requirements

- Maintain a searchable database of clients and their event history.
- Schedule events with accurate dates, budgets, venues, and statuses.
- Manage vendors and track which vendors are engaged for which event.
- Collect and analyze client feedback after every event.
- Prevent invalid data such as past event dates and zero or negative budgets.
- Notify responsible staff and clients before an event takes place.
- Control and approve high-value event budgets.
- Restrict access to sensitive information based on user responsibility.
- Provide management with visual insight into upcoming events and budgets.

2.1.2 Requirement-to-Salesforce Mapping

Business Requirement	Salesforce Capability Used
Store clients, vendors, venues, events	Custom Objects and Custom Fields
Link events to clients, venues, and vendors	Lookup and Master-Detail Relationships, Junction Object
Different handling for event categories	Record Types and Page Layouts
Prevent incorrect dates and budgets	Validation Rules
Automatic event reminders	Scheduled-Triggered Flow with Email Alerts
Budget sign-off	Approval Process
Complex cross-object updates	Apex Triggers and Handler Classes
Data privacy	Profiles, Roles, Permission Sets, Sharing Rules
Business insight	Reports and Lightning Dashboards

2.1.3 Planning Approach

The project followed an iterative build approach. The data model was designed first because every other component depends on it. Security was then layered over the data model, followed by automation, and finally reporting. Each phase was tested before the next phase began. Declarative tools were preferred, and Apex was reserved for requirements that could not be satisfied with Flow or validation rules, in line with Salesforce best practice.

2.2 Phase 2: Data Model and Customization

The data model is the foundation of the EventForce system. It was designed to model real-world event planning relationships while keeping the schema simple, normalized, and reportable.

2.2.1 Custom Objects

Object	Purpose	Key Relationships
Client	Stores organizations or individuals who book events	Parent of Events (Lookup)
Event	Central object representing a planned event	Lookup to Client and Venue; parent of Event Feedback and Event Vendor
Vendor	Stores catering, decoration, sound, photography, and other service providers	Parent of Event Vendor (Master-Detail)
Venue	Stores locations where events can be held	Referenced by Event (Lookup)
Event Vendor	Junction object connecting Events and Vendors (many-to-many)	Master-Detail to Event and to Vendor
Event Feedback	Stores ratings and comments submitted after an event	Master-Detail to Event

2.2.2 Custom Fields

The principal custom fields designed for each object are summarized below.

Object	Field	Data Type	Description
Event	Event Date	Date	Scheduled date of the event
Event	Event Budget	Currency	Approved or proposed budget for the event
Event	Status	Picklist	Planned, Confirmed, In Progress, Completed, Cancelled
Event	Approval Status	Picklist	Draft, Submitted, Approved, Rejected
Event	Expected Attendees	Number	Estimated number of guests
Client	Email, Phone	Email, Phone	Primary contact details used for notifications
Client	Client Type	Picklist	Individual, Corporate, Institution
Vendor	Service Category	Picklist	Catering, Decoration, Photography, Sound and Lighting, Other
Vendor	Contact Email, Phone	Email, Phone	Vendor contact information
Venue	Capacity, Location	Number, Text	Seating capacity and address of the venue
Event Vendor	Agreed Cost, Status	Currency,	Cost and engagement status of a vendor

Object	Field	Data Type	Description
		Picklist	for an event
Event Feedback	Rating, Comments	Number, Long Text	Rating from 1 to 5 and written feedback

2.2.3 Relationships

- **Lookup Relationships:** An Event references one Client and one Venue through Lookup relationships. A Lookup is used because a Client or Venue must continue to exist independently even if an Event is deleted.
- **Master-Detail Relationship:** Event Feedback is a child of Event through a Master-Detail relationship. Feedback has no meaning without its event, so it inherits sharing and is deleted along with the parent. This also enables roll-up summary fields such as the average rating of an event.
- **Junction Object:** Event Vendor is a junction object with two Master-Detail relationships, one to Event and one to Vendor. This allows many vendors to work on one event and one vendor to work on many events.

2.2.4 Entity Relationship Diagram

[INSERT ENTITY RELATIONSHIP DIAGRAM HERE]

Suggested source: Schema Builder (Setup > Schema Builder), screenshot of Client, Event, Vendor, Venue, Event Vendor, and Event Feedback.

Figure 1: Entity Relationship Diagram of the EventForce Data Model

2.2.5 Record Types

Record Types allow different categories of events to follow different picklist values and page layouts while sharing the same object. The Event object uses the following record types:

- **Corporate Event:** For business conferences, seminars, product launches, and corporate celebrations. Emphasizes budget, attendee count, and vendor assignments.
- **Wedding Event:** For weddings and receptions. Emphasizes venue, guest count, decoration, and catering.
- **Private / Social Event:** For birthdays, anniversaries, and family functions. Uses a simplified layout with the essential fields only.

2.2.6 Page Layouts

A dedicated page layout is assigned to each record type and user profile. Layouts are organized into logical sections such as Event Information, Financial Details, Venue and Schedule, and System Information. Related lists for Event Vendors and Event Feedback are placed on the Event record page, and the Lightning Record Page includes a highlights panel so that status and date are visible at a glance. Sensitive financial fields are removed or made read-only on layouts assigned to junior staff.

2.3 Phase 3: Business Automation

Phase 3 introduced business logic into the system. Automation was implemented in a deliberate order: validation rules to protect data quality, Flow for routine communication, Approval Processes for governance, and Apex for logic that declarative tools cannot handle.

2.3.1 Validation Rules

Validation rules prevent invalid data from being saved and display a clear message to the user.

Rule Name	Object	Formula	Error Message
Event_Date_Not_In_Past	Event	AND(ISNEW(), Event_Date__c < TODAY())	Event date cannot be in the past.
Event_Budget_Must_Be_Positive	Event	AND(NOT(ISBLANK(Event_Budget__c)), Event_Budget__c <= 0)	Event budget must be greater than zero.
Budget_Required_When_Confirmed	Event	AND(ISPICKVAL(Status__c, "Confirmed"), ISBLANK(Event_Budget__c))	A budget is required before an event is confirmed.
Feedback_Rating_Range	Event Feedback	OR(Rating__c < 1, Rating__c > 5)	Rating must be between 1 and 5.

The ISNEW() condition in the date rule ensures that existing events can still be edited after their date has passed, for example when marking an event as Completed.

2.3.2 Flow: 3-Day Event Reminder

The 3-Day Event Reminder is a Scheduled-Triggered Flow that runs once every day and notifies stakeholders three days before each event.

Flow Design

1. **Start:** The flow starts on a daily schedule.
2. **Entry Criteria:** Event records where Event Date equals three days from today and Status is not Cancelled or Completed are selected.
3. **Get Records:** For each qualifying event, the flow retrieves the related Client and the Event Owner.
4. **Action:** An email alert using a standard template is sent to the client contact and to the event owner, containing the event name, date, venue, and status.
5. **Follow-up:** A Task is created for the event owner to confirm vendor readiness and venue preparation.
6. **Error Handling:** A Fault Path is attached to the email and task elements so that any failure is captured and the error details are emailed to the administrator.

A scheduled flow was chosen instead of a record-triggered flow because the trigger is the passage of time and not a change to a record. The flow design is also bulk-safe, since DML and email actions are performed outside loops.

2.3.3 Approval Process: Event Budget Approval

To ensure financial control, events whose budget exceeds a configurable threshold must be approved before they can be confirmed.

Component	Configuration
Object	Event
Entry Criteria	Event Budget is greater than the defined threshold (for example, 5,000) and Approval Status is Draft or Rejected
Submitter	Event Coordinator (record owner)
Step 1	Approval by the Event Manager (manager in the role hierarchy)
Step 2 (optional)	Approval by the Finance Approver queue for budgets above the higher threshold
On Submission	Record locked, Approval Status set to Submitted
On Final Approval	Approval Status set to Approved, record unlocked, notification email sent to the submitter
On Rejection	Approval Status set to Rejected, record unlocked, rejection comments returned to the submitter

The record is locked during approval so that the budget cannot be altered after submission. Approvers can respond from Salesforce, from the notification email, or from the mobile application.

2.3.4 Apex Triggers and Logic

Apex was used for logic that involves cross-object processing and cannot be expressed with validation rules or Flow alone. The implementation follows the one-trigger-per-object pattern, in which the trigger contains no business logic and delegates to a handler class. This keeps the code testable, maintainable, and free from conflicting triggers.

The trigger on the Event object performs two responsibilities:

- Before insert and update: confirms that a venue is assigned before an event is moved to Confirmed, and prevents the date of a Completed event from being changed.
- After update: when an event is cancelled, all related Event Vendor assignments are automatically marked as Cancelled so that vendors are not left committed to an event that no longer exists.

Trigger

```
trigger EventTrigger on Event__c (before insert, before update, after update) {
    if (Trigger.isBefore) {
        EventTriggerHandler.validateEvents(Trigger.new, Trigger.oldMap);
    }
    if (Trigger.isAfter && Trigger.isUpdate) {
        EventTriggerHandler.handleCancellations(Trigger.new, Trigger.oldMap);
    }
}
```

Handler Class

```
public with sharing class EventTriggerHandler {

    public static void validateEvents(List<Event__c> newEvents, Map<Id,
Event__c> oldMap) {
        for (Event__c ev : newEvents) {
            Event__c oldEv = (oldMap == null) ? null : oldMap.get(ev.Id);
            if (ev.Status__c == 'Confirmed' && ev.Venue__c == null) {
                ev.addError('A venue must be assigned before an event can be
confirmed.');
            }
            if (oldEv != null && oldEv.Status__c == 'Completed'
                && ev.Event_Date__c != oldEv.Event_Date__c) {
                ev.Event_Date__c.addError('The date of a completed event cannot
be changed.');
            }
        }
    }

    public static void handleCancellations(List<Event__c> newEvents, Map<Id,
Event__c> oldMap) {
        Set<Id> cancelledIds = new Set<Id>();
        for (Event__c ev : newEvents) {
            Event__c oldEv = oldMap.get(ev.Id);
            if (ev.Status__c == 'Cancelled' && oldEv.Status__c != 'Cancelled')
{
                cancelledIds.add(ev.Id);
            }
        }
        if (cancelledIds.isEmpty()) { return; }

        List<Event_Vendor__c> assignments = [
            SELECT Id, Status__c FROM Event_Vendor__c
            WHERE Event__c IN :cancelledIds AND Status__c != 'Cancelled'
        ];
        for (Event_Vendor__c a : assignments) { a.Status__c = 'Cancelled'; }
        if (!assignments.isEmpty()) { update assignments; }
    }
}
```

Design Considerations

- All logic is bulkified. Queries and DML statements are executed once per transaction and never inside loops, which protects against governor limits during data loads.
- The class uses the with sharing keyword so that user permissions are respected.
- An Apex test class covers single-record and bulk scenarios (200 records), positive and negative paths, and asserts the expected outcomes. Salesforce requires a minimum of 75 percent code coverage for production deployment.

2.4 Phase 4: Security, Access Controls and Analytics

2.4.1 Security Model

Security in Salesforce is layered. The EventForce security design applies the principle of least privilege, in which users receive only the access they need, starting from a restrictive baseline and opening access where required.

Organization-Wide Defaults (OWD)

Object	Default Access	Reason
Event	Private	Events contain budget information and should be visible only to owners and their managers
Client	Private	Client contact data is confidential
Vendor	Public Read Only	All planners need to view vendor details to make selections
Venue	Public Read Only	Venue information is shared reference data
Event Vendor	Controlled by Parent	Master-Detail child; inherits access from Event
Event Feedback	Controlled by Parent	Master-Detail child; inherits access from Event

Roles

A role hierarchy gives managers visibility into the records owned by their subordinates:

- Operations Director (top of hierarchy, sees all data)
- Event Manager (sees records of all coordinators beneath)
- Event Coordinator (sees own records)

Profiles

Profile	Intended Users	Access Summary
System Administrator	Project administrators	Full configuration and data access
Event Manager	Senior planners	Create, read, edit, delete on Events; approve budgets
Event Coordinator	Day-to-day planners	Create, read, edit on Events, Clients, Event Vendors; no delete
Read Only / Viewer	Auditors, trainees	Read-only access to permitted objects

Permission Sets

Permission sets extend access without creating additional profiles. The following sets were designed:

- **Budget Approver:** Grants access to the Event Budget field and the ability to approve budgets.
- **Vendor Management:** Grants create and edit access to Vendor and Event Vendor records.
- **Reports and Dashboards Access:** Grants permission to create and edit reports and dashboards.

Field-Level Security

Event Budget and Agreed Cost fields are visible only to Event Managers and users holding the Budget Approver permission set. Coordinators have read-only access to these fields where operationally required.

Sharing Rules

- A criteria-based sharing rule shares Corporate Events with the Finance Approver public group on a read-only basis so that finance staff can review them without owning them.
- An owner-based sharing rule shares Events owned by the Coordinator role with the Event Manager role, supporting cross-team collaboration beyond the strict hierarchy.

2.4.2 Reports and Dashboards

Analytics give management immediate visibility into operations. Custom Report Types were created to join related objects, allowing questions that span multiple objects to be answered in a single report.

Report	Report Type	Business Question Answered
Upcoming Events by Month	Events	How many events and how much budget is scheduled in each month?
New Events with Venue Report	Events with Venues	How many new events have a venue assigned, by month?
Events by Status	Events	How many events are Planned, Confirmed, Completed, or Cancelled?
Vendor Utilization	Events with Event Vendors	Which vendors are engaged most frequently?
Client Event History	Clients with Events	What events has each client booked?
Feedback Summary	Events with Feedback	What is the average rating for events and which events score poorly?

Eventforce Operation Dashboard

The Eventforce Operation Dashboard is built inside the Event Planner application and presents operational data visually. It currently contains two widgets, each linked to an underlying report through a View Report link so that users can drill down into the detailed records.

- **Upcoming Events by Month:** A donut chart that displays the Sum of Event Budget grouped by Event Date by month. In the screenshot below, it shows a total budget of $10K scheduled for October 2026.
- **New Events with Venue Report:** A funnel chart based on the New Events with Venue Report. It displays the record count of new events that have a venue assigned, grouped by Event Date month. In the screenshot below, it shows a record count of 1 for October 2026.

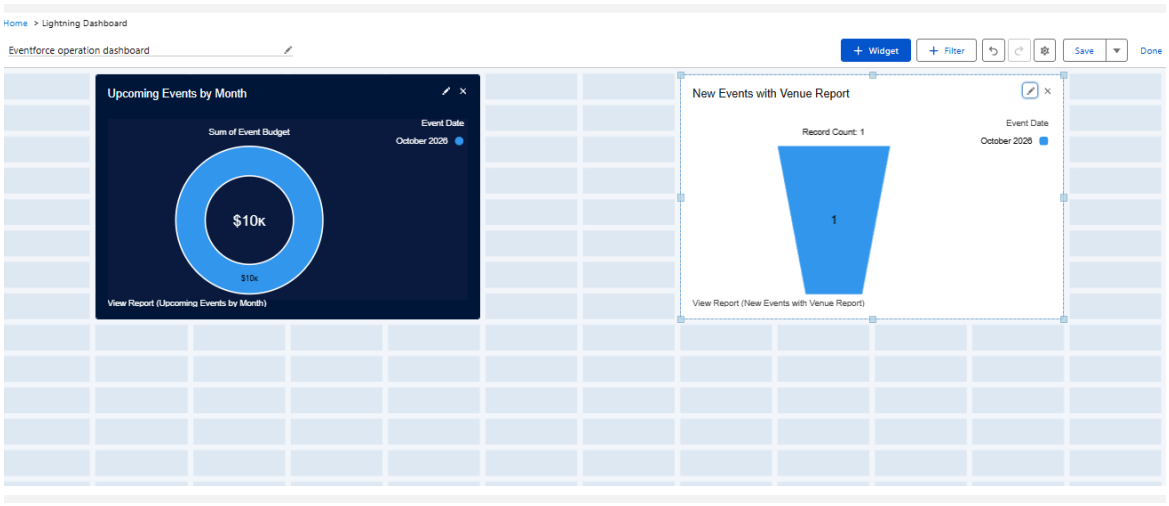


Figure 2: Eventforce Operation Dashboard with the Upcoming Events by Month and New Events with Venue Report widgets

The dashboard is designed to be extended with additional widgets drawn from the reports listed above, such as Events by Status, Vendor Utilization, and Average Feedback Rating, so that management obtains a complete operational view from one screen. Dashboard running-user and folder access are configured so that viewers see data in line with the sharing model.

3. PHASE 5: DEPLOYMENT, MONITORING AND TROUBLESHOOTING

3.1 Maintenance and Monitoring

A system is only valuable if it continues to work reliably after it is built. Monitoring is therefore performed proactively, so that failed automations are detected before users or clients are affected.

3.1.1 Monitoring Apex Jobs

- The Apex Jobs page (Setup > Environments > Jobs > Apex Jobs) is reviewed to confirm that asynchronous and scheduled Apex completes successfully.
- Failed jobs are investigated using the status details, number of errors, and the associated error message.
- Apex Exception Emails are enabled so that unhandled exceptions are sent to the administrator.

3.1.2 Debug Logs

- Trace flags are set for specific users with an appropriate log level and a short expiry.
- Logs are analyzed for SOQL queries, DML operations, governor limit usage, and the exact line where an exception occurred.
- The Developer Console and the Log Inspector are used to follow the execution order of triggers, flows, validation rules, and workflows within a transaction.

3.1.3 Flow Interviews

- The Paused and Failed Flow Interviews page is reviewed to identify runs that did not complete.
- Flow fault paths send error details by email, including the flow name, record identifier, and error text.
- Flow Debug and the Run history are used to trace decisions and element outputs when a flow produces an unexpected result.

3.1.4 Routine Maintenance

- Periodic review of storage usage and API usage in the Company Information page.
- Review of the Setup Audit Trail to track configuration changes and identify the source of unexpected behavior.
- Regular review of user accounts, deactivation of departed users, and adjustment of permission sets.
- Data quality checks for duplicates and incomplete records, supported by duplicate and matching rules where appropriate.

3.2 Troubleshooting Approach

Troubleshooting follows a consistent method: reproduce the issue, identify the exact component involved, determine the root cause, apply a corrective change, and verify the result. Typical scenarios are described below.

Issue	Typical Root Cause	Resolution
User cannot see a record	OWD is Private and no role or sharing rule grants access	Review role placement, add or adjust a sharing rule, or use manual sharing for a specific exception
User cannot see or edit a field	Field-level security or page layout excludes the field	Update field-level security or assign the correct permission set and layout
"Insufficient privileges" on save	Missing object permission or a conflicting profile and permission set combination	Compare effective permissions using the user's profile and permission sets; grant the minimum required permission
Record cannot be saved	A validation rule is blocking the save	Review the error message, check the rule formula and entry conditions, and correct the data or revise the rule
Reminder emails not sent	Flow inactive, entry criteria wrong, or email deliverability disabled	Check flow status and version, test entry criteria in Debug, and verify Email Deliverability is set to All Email
Trigger throws an exception	Unhandled null value, bulk data, or governor limit exceeded	Reproduce with a debug log, add null checks, bulkify queries and DML, and extend the test class to cover the case
Record stuck in approval	Approver unavailable or approval step misconfigured	Reassign the approval request or recall it, and review approver and delegation settings

As business needs evolve, validation rules, sharing rules, and permission sets are revisited and updated. Every change is first tested in a sandbox or a test user context, documented, and then released, so that fixes do not introduce new defects.

3.3 Deployment Strategy

The EventForce system was developed in a Developer Edition Org. Moving it to an Enterprise Edition production environment requires a disciplined release process.

3.3.1 Environment Strategy

1. **Development:** Development: build and unit test in a Developer Org or Developer Sandbox.
2. **Testing / UAT:** Testing: deploy to a Partial Copy or Full Sandbox containing realistic data for integration testing and User Acceptance Testing.
3. **Production:** Production: deploy to the Enterprise Edition production org after approval.

3.3.2 Deployment Tools

A Developer Edition Org has no sandbox relationship with a production org, so change sets cannot be used directly from it. The metadata is therefore retrieved from the Developer Org using the Salesforce CLI and the Metadata API (using a package.xml manifest or source-format project), stored in a Git repository, and deployed to the target environment. Between sandboxes and production, Change Sets, Salesforce CLI, or DevOps Center may be used.

3.3.3 Recommended Deployment Order

1. Custom objects, fields, and relationships
2. Record types, page layouts, and Lightning record pages
3. Validation rules
4. Apex classes and triggers, together with test classes
5. Flows and email templates
6. Approval processes
7. Permission sets, profiles, and sharing rules
8. Reports, report types, and dashboards
9. Reference data and initial records, loaded with Data Loader or the Data Import Wizard

3.3.4 Best Practices

- Maintain all metadata in version control so that every change is traceable and reversible.
- Run all local Apex tests during validation and ensure at least 75 percent code coverage, with meaningful assertions.
- Perform a validation-only deployment (check-only) before the actual release to detect errors early.
- Deploy during a low-usage window and communicate the release schedule to users.
- Prepare a rollback plan, including a metadata backup of the previous version and a data backup where relevant.
- Activate flows and approval processes only after post-deployment verification, since deployed flows may need to be activated manually depending on the deployment settings.
- Complete a post-deployment checklist covering user access, automation, reports, dashboards, and email deliverability.
- Provide user training and short guides so that staff can adopt the new system confidently.

4. CONCLUSION AND PROJECT LEARNINGS

4.1 Summary of Project Outcomes

The EventForce Management System demonstrates how the Salesforce platform can be used to build a complete, real-world CRM solution. The project delivered a normalized data model with six related objects, record types and page layouts for different event categories, validation rules that protect data quality, a scheduled flow that automates event reminders, an approval process that governs event budgets, and Apex logic that maintains consistency across related records. A layered security model enforces data privacy, and the Eventforce Operation Dashboard provides visual insight for decision makers.

Together, these components replace manual coordination with a structured, automated, and secure process. Event information is centralized, deadlines are communicated automatically, financial decisions are controlled, and management can monitor operations in real time. The project also confirms that most business requirements can be solved declaratively, with code reserved for the few cases that need it.

4.2 Key Takeaways

- **System Architecture:** Planning the data model first and mapping each requirement to the correct Salesforce feature avoided rework and kept the design clean.
- **Scalability:** Using Master-Detail relationships, junction objects, and a one-trigger-per-object pattern created a structure that is easy to extend with new objects, record types, and rules.
- **Stability:** Bulk-safe automation, fault paths in flows, error handling in Apex, and test coverage make the system dependable under real usage.
- **Security:** Starting with restrictive defaults and opening access through roles, permission sets, and sharing rules is more maintainable than granting broad access and restricting it later.
- **Platform Discipline:** Awareness of governor limits, order of execution, and bulkification is essential for writing production-quality Apex and Flow.
- **Operations:** Debug logs, Apex Jobs, and Flow interview monitoring turn failures into quickly diagnosable events instead of silent problems.

4.3 Future Enhancements

- Experience Cloud portal so that clients can view event status and submit feedback online.
- Integration with calendar and payment services for scheduling and invoicing.
- Additional dashboard widgets for vendor performance, budget versus actual spending, and customer satisfaction trends.
- Lightning Web Components for a customized event timeline and vendor selection experience.
- SMS or WhatsApp reminders in addition to email notifications.

4.4 Final Remarks

The EventForce Management System proves that a Developer Org, when combined with sound design principles, can be used to build a professional-grade CRM application that follows industry practices in data modelling, automation, security, analytics, and release management. The skills gained through this project provide a strong foundation for developing and administering enterprise solutions on the Salesforce platform.